



**UNIVERSIDAD AUTONOMA DE  
NUEVO LEON**

**FACULTAD DE CIENCIAS FISICO-  
MATEMATICAS**



**LICENCIATURA EN CIENCIAS COMPUTACIONALES**

**LABORATORIO DE PROGRAMACION ORIENTADA A OBJETOS**

## **MANUAL DE TECNICO**

**Docente:** José Anastacio Hernández Saldaña

**Nombre:** Aldo Gabriel Gómez Canizales

**Matrícula:** 1923886

**Fecha:** viernes 22 de mayo de 2026

## **MANUAL DE TECNICO**

### **Sistema de Punto de Venta para Gimnasio (ActiveLife)**

---

#### **SECCION 1. INTRODUCCION AL SISTEMA**

Bienvenido al Manual de Usuario de ActiveLife. Este software fue diseñado con el propósito de automatizar tareas administrativas comunes dentro de un gimnasio, permitiendo gestionar:

- Clientes
- Membresías
- Inventario de equipo
- Calendario de clases grupales

La aplicación busca proporcionar una interfaz sencilla, modular y fácilmente mantenible utilizando el patrón MVC (Modelo-Vista-Controlador).

#### **SECCION 2. INTRODUCCION AL MANUAL**

Este documento está dirigido exclusivamente a desarrolladores y personal técnico encargado del mantenimiento, mejora o extensión del software ActiveLife.

El manual describe:

- Arquitectura general
- Tecnologías utilizadas
- Estructura del proyecto
- Responsabilidades de clases
- Persistencia de datos
- Diagramas UML
- Validaciones implementadas
- Flujo de navegación
- Posibles mejoras futuras

El objetivo es facilitar la comprensión interna del sistema y permitir futuras modificaciones sin afectar la estabilidad de la aplicación.

### SECCION 3. TECNOLOGIAS UTILIZADAS

La aplicación fue desarrollada utilizando las siguientes tecnologías:

**Java.** Lenguaje principal utilizado para la lógica de negocio y programación orientada a objetos. Se implementaron conceptos fundamentales de POO:

- Encapsulamiento
- Clases y objetos
- Modularidad
- Separación de responsabilidades

**JavaFX.** Framework gráfico utilizado para construir la interfaz de usuario. JavaFX permitió implementar:

- Ventanas
- Tablas
- Formularios
- Botones
- Menús
- Navegación dinámica

**FXML.** Se utilizó para separar la estructura visual de la lógica del programa. Las ventajas del uso de FXML fueron:

- Separación MVC
- Mayor mantenibilidad
- Mejor organización visual
- Compatibilidad con Scene Builder

**CSS.** La aplicación utiliza un archivo CSS nombrado 'gym\_style.css'. Este archivo proporciona:

- Colores
- Tipografía
- Estilo de botones
- Estilo de tablas
- Apariencia moderna y uniforme

**Netbeans IDE.** El proyecto fue desarrollado en NetBeans IDE debido a:

- Integración sencilla con JavaFX
- Gestión automática de proyectos Java
- Compatibilidad con Scene Builder
- Herramientas de depuración

**Resumen:** Como objetivo principal del curso de aprendizaje, se usó el lenguaje **Java/JavaFX** para echar a la práctica los conocimientos adquiridos en POO (Programación Orientado a Objetos). También se usó el IDE de **Netbeans** para construir el software, aunque también se dió la libertad de usar otros IDEs populares. Como el proyecto se tuvo que integrar en un esquema MVC (Modelo-Vista-Controlador), el cual fue también un fundamento a interiorizar en el curso y que simplemente expresa que hay que separar dichos elementos de tal manera que funcionen independientemente (modificar uno no implica hacer cambios en el otro), se usó **FXML** para el layout (estructura) y **CSS** para el aspecto visual del software.

## SECCION 4. ARQUITECTURA GENERAL

La aplicación utiliza una arquitectura basada en el patrón MVC (Modelo-Vista-Controlador). Está conformada por layouts FXML con su propio controlador Java. Utiliza un solo stylesheet CSS para dar aspecto visual a todos los layouts FXML.

**Modelo.** Representa la lógica y los datos del sistema. Está conformada por las siguientes clases:

- Cliente
- Usuario
- Membresia
- Equipo

**Vista.** Representada por archivos FXML, los cuales son:

- mainLayout.fxml
- menuClientes.fxml
- menuMembresias.fxml
- menuInventario.fxml
- menuCalendario.fxml
- menuLogin.fxml
- MenuUsuario.fxml

**Controlador.** Cada layout FXML tiene un controlador Java asociado. Por ejemplo:

- MenuClientes.fxml -> MenuClientes.java
- MenuMembresias.fxml -> MenuMembresias.java
- MenuInventario.fxml -> MenuInventario.java

## SECCION 5. CLASES Y RESPONSABILIDADES

La aplicación se conforma de las siguientes clases con sus responsabilidades:

**Main.** Clase principal del sistema.

### **Responsabilidades:**

- Punto de entrada
- Manejo de ventanas
- Carga de FXML
- Lectura/escritura serializada de clientes
- Validaciones auxiliares

Funciones:

- loadPage()
- guardaClientes()
- leeClientes()
- validaCampoNumerico()

**MainLayout.** Controlador principal de navegación.

### **Responsabilidades:**

- Cambiar entre menús
- Controlar acceso según sesión iniciada

**Cliente.** Representa a un cliente del gimnasio.

### **Atributos:**

- ID
- Nombre completo
- Email
- Membresía asignada

**Usuario.** Representa usuarios administrativos.

**Atributos:**

- ID
- Nombre completo.

**Membresia.** Representa los planes de un cliente.

**Atributos:**

- Tipo
- Fecha de inicio
- Duración (en días)
- Estado (activo o fuera de servicio).

**Equipo.** Representa un equipo del gimnasio.

**Atributos:**

- ID
- Nombre descriptivo
- Estado (Activo o fuera de servicio)

**MenuLogin.** Controla autenticación.

**Responsabilidades:**

- Iniciar sesion
- Validar acceso a solo administradores del gimnasio

**MenuClientes.** Controlador CRUD de clientes.

**Responsabilidades:**

- Crear cliente
- Modificar cliente
- Eliminar cliente
- Mostrar tabla de clientes registrados

**MenuMembresias.** Gestiona membresías.

**Responsabilidades:**

- Asignar membresías
- Renovar membresías
- Mostrar estado de las membresías de clientes registrados
- Mostrar cuándo vencen las membresías de clientes registrados

**MenuInventario.** Gestiona inventario.

**Responsabilidades:**

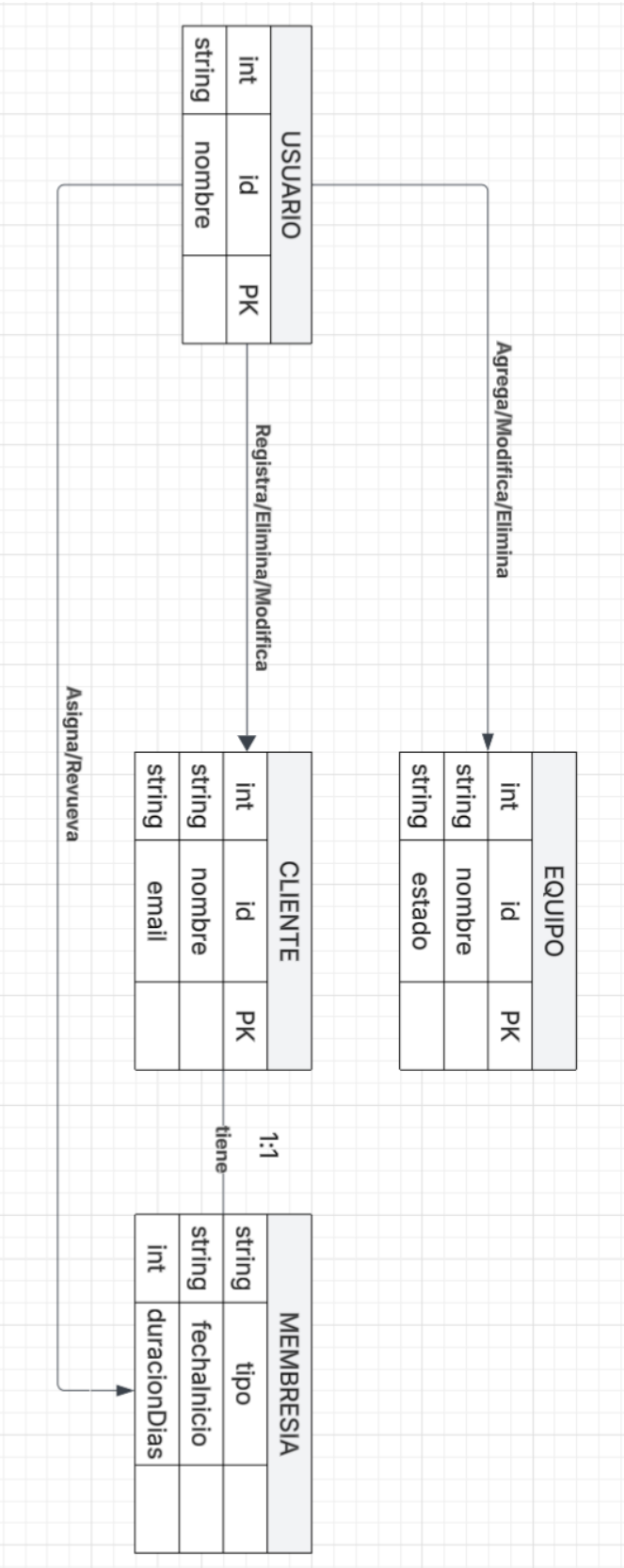
- Registrar equipo
- Modificar registros del equipo
- Eliminar registros del equipo

**MenuCalendario.** Muestra calendario de las actividades diarias del gimnasio.

**MenuUsuario.** Muestra datos del usuario actual.



SECCION 6. DIAGRAMA UML GENERAL DE LA APLICACION



## SECCION 7. PERSISTENCIA

La aplicación guarda los datos de los clientes en un archivo serial nombrado **'clientes.ser'**. Las funciones **'guardaClientes'** y **'leeClientes'** hacen uso de este archivo para hacer sus operaciones.

## SECCION 8. POSIBLES MEJORAS

A continuación, se presenta una lista de mejoras que se podrían implementar en la aplicación que la permitan hacer más **robusta, simple y completa**:

**Mejor aspecto visual.** Crear un mejor diseño que permita a la aplicación tener un interfaz más responsivo y con un estilo moderno.

**Reportes estadísticos.** Muestra un resumen estadístico de los clientes e inventario.

**Validación de los campos de correo electrónico.** Que siga cualquiera de los siguientes patrones: ...@email.com, ...@sample.com

**Asignación/Revocación de permisos de los usuarios.** Los usuarios pueden leer y/o modificar la base de datos de los clientes, membresías e inventario.

**Búsqueda de usuarios, clientes o equipos por índice.** El ID del usuario, cliente o equipo es prácticamente el índice en el cual el elemento se ubica en el arreglo. Esto simplificaría la codificación al rechazar las funciones de búsqueda por la indexación (que es más rápido y directo).